

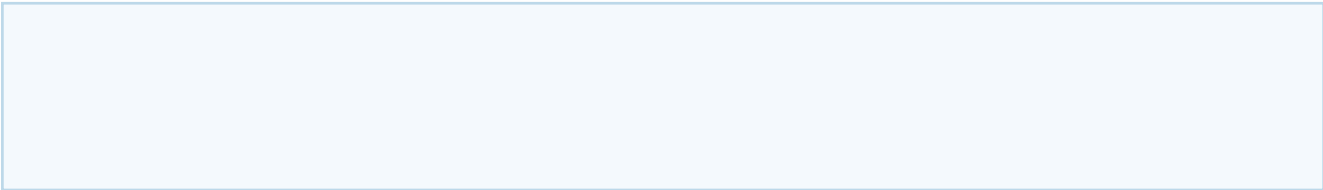
STAYORA HOTEL BOOKING

Full-Stack Web Application — Comprehensive Interview Guide

React (Vite) + Node.js + Express.js + PostgreSQL + OpenStreetMap

Table of Contents

- 1. Executive Project Summary (Elevator Pitch)
- 2. Technical Architecture & Tech Stack Explained
- 3. Page-by-Page Breakdown & User Journey
- 4. Database Design & PostgreSQL Schema
- 5. Backend API Endpoints & CRUD Architecture
- 6. Frontend Components, Hooks & State Management
- 7. **Target Audience / Purpose of This Guide:**
This guide is specifically tailored for fresher & high-seniority developers. It explains every basic concept from scratch without confusing jargon, giving you ready-to-speak interview answers so you can confidently explain the project.
- 8. OpenStreetMap & Location Services Integration
- 9. Non-Technical / HR Questions & Tips for Success



1. Executive Project Summary (Elevator Pitch)

Stayora is a modern, responsive Full-Stack Hotel Booking & Management Web Application. It enables guests to search for luxury hotels across top Indian destinations (Kerala, Tamil Nadu, Goa, Karnataka), view interactive maps with GPS coordinates, explore rooms, and complete bookings. Furthermore, it features a dedicated CRUD Hotel Management portal allowing administrators to Create, View, Update, and Delete hotels in real-time.

How to Introduce the Project in an Interview (Say this!):

"I built 'Stayora', a full-stack hotel booking platform using React for the frontend, Node.js and Express for the REST API backend, and PostgreSQL for persistent relational data storage. The application features complete CRUD management for hotels, dynamic search and filtering by destination, room booking workflows, and embedded interactive OpenStreetMap locations based on real latitude and longitude coordinates."

Core Features at a Glance:

- **Destination Search & Filter:** Guests can filter hotels by destination (Kerala, Tamil Nadu, Goa, Karnataka) or search by name/city.
- **Hotel Showcase:** Displays price per night, star rating, room availability, amenities, and high-resolution visuals.
- **Interactive Map Integration:** Clicking 'View Details' opens a dedicated page with an OpenStreetMap embed showing the hotel's exact GPS marker.
- **Full CRUD Management:** A dedicated admin page (/manage-hotels) and on-card actions to Create, Read, Update, and Delete hotel listings.
- **Reservation System:** Guests select rooms, input traveler details, choose payment options, and receive confirmed booking IDs.

2. Technical Architecture & Tech Stack

The project follows the standard Client-Server 3-Tier Architecture:

- **1. Presentation Layer (Frontend):** Built using React 18 and Vite. Handles the user interface, routing, forms, client-side validation, and makes asynchronous HTTP fetch requests to the backend.
- **2. Application Layer (Backend):** Built using Node.js and Express.js. Implements RESTful API endpoints, handles business logic, request validation, CORS, error handling, and queries the database.
- **3. Database Layer (PostgreSQL):** PostgreSQL relational database running with connection pooling via the 'pg' library. Stores normalized tables for hotels, rooms, locations, and bookings.

Why These Technologies Were Chosen:

- **React (Vite):** Component-based architecture allows reusable UI elements (Navbar, HotelCard, RoomCard). Fast bundle building with Vite.
- **Node.js & Express:** Single-threaded asynchronous event loop enables handling concurrent API requests efficiently using JavaScript on both client and server.
- **PostgreSQL:** Reliable ACID-compliant relational database. Supports strong schema integrity, foreign keys, array data types (amenities), and numeric coordinates.
- **OpenStreetMap:** Free, open-source mapping service with no expensive API keys or billing quotas required.

3. Page-by-Page Breakdown & User Journey

1. Home Page (Route: '/' and '/home')

- File: frontend/src/pages/home.jsx
- Features a full-width background hero video with overlay, branding typography, and a destination dropdown.
- The dropdown dynamically fetches destinations from backend GET /api/locations/destinations ('Kerala', 'Tamil Nadu', 'Goa', 'Karnataka').
- Selecting a destination and clicking 'Show hotels' navigates to /hotels passing destination in React Router state.

2. Hotels Catalog Page (Route: '/hotels')

- File: frontend/src/pages/hotel.jsx
- Displays verified hotel cards matching the search term or selected destination.
- Each card shows thumbnail, city, state, price, star rating, amenities badges, and a mini-map embed.
- Actions on each card: 'View Details' (opens /hotels/:id), 'Book Now' (opens /register), 'Edit' (pre-fills /manage-hotels form), and 'Delete' (deletes hotel with confirmation).

3. Hotel Details Page (Route: '/hotels/:id')

- File: frontend/src/pages/HotelDetails.jsx
- Fetches specific hotel details using GET /api/hotels/:id using the ID parameter from useParams().
- Embeds an interactive OpenStreetMap iframe using the hotel's exact latitude and longitude.
- Lists room options (Standard, Deluxe, Suite) with facilities and 'Book Now' triggers.
- Includes quick 'Edit Hotel' and 'Delete Hotel' admin actions directly on the page.

4. Manage Hotels CRUD Page (Route: '/manage-hotels')

- File: frontend/src/pages/ManageHotels.jsx
- Create Form: Inputs for Name, State, City, Destination, Price, Rooms, Rating, Latitude, Longitude, Amenities, Description, and Image upload (up to 5MB).
- Read / Table View: Displays all hotels currently in PostgreSQL with thumbnails, GPS chips, and action buttons.
- Update: Clicking 'Edit' pre-populates all inputs, switches the button to 'Update Hotel', and executes PUT /api/hotels/:id.
- Delete: Confirms deletion and issues DELETE /api/hotels/:id, updating the table state instantaneously.

5. Register & Booking Confirmation (Routes: '/register', '/confirmation', '/bookings')

- Register Page (register.jsx): Traveler details (Name, Email, Phone, Check-in, Check-out, Guests, Payment Method).
- Confirmation Page (confirmation.jsx): Renders booking receipt with generated booking ID (e.g. STAY-XXXX).
- My Bookings Page (bookings.jsx): Lists all confirmed reservations with status and cancellation options.

4. Database Design & PostgreSQL Schema

The database schema is defined in backend/src/db/schema.sql and managed via PostgreSQL:

1. 'hotels' Table

- id SERIAL PRIMARY KEY: Unique auto-incrementing integer identifier.
- name VARCHAR(255) NOT NULL: Name of the resort/hotel.
- city VARCHAR(100), state VARCHAR(100), destination VARCHAR(100): Geographic classification.
- price NUMERIC(10, 2) NOT NULL: Nightly rate in INR.
- rooms INT DEFAULT 10: Available room inventory.
- rating NUMERIC(3, 1): Rating score (e.g., 4.8).
- image TEXT: URL or base64 data string of the hotel photo.
- description TEXT: Descriptive overview.
- amenities TEXT[]: PostgreSQL array of amenities (e.g., {'Free Wi-Fi', 'Breakfast'}).
- latitude NUMERIC(10, 6), longitude NUMERIC(10, 6): Precise GPS coordinates.

2. 'rooms' Table

- hotel_id INT REFERENCES hotels(id) ON DELETE CASCADE: Foreign key linked to hotels table.
- name, price, available, image, facilities TEXT[].

3. 'locations' Table

- state VARCHAR(100) UNIQUE, cities TEXT[]: Supported Indian states and city lists.

4. 'bookings' Table

- booking_id VARCHAR(60) UNIQUE, full_name, email, phone, selected_hotel, selected_room, check_in, check_out, guests, total_price, status.

5. Backend REST API Endpoints

The backend Express server exposes standard RESTful endpoints returning JSON responses:

- **Health Check:** GET /api/health — Checks backend uptime and PostgreSQL database connection status.
- **Read All Hotels:** GET /api/hotels — Retrieves all hotels; accepts query filters: ?destination=Goa, ?search=Kochi, ?minPrice=1000.
- **Read Single Hotel:** GET /api/hotels/:id — Retrieves complete profile of a single hotel by its primary key.
- **Create Hotel:** POST /api/hotels — Creates a new hotel record. Validates required fields (name, city, price, rooms).
- **Update Hotel:** PUT /api/hotels/:id — Updates an existing hotel's fields (name, price, coordinates, etc.).
- **Delete Hotel:** DELETE /api/hotels/:id — Deletes a hotel record from database.
- **Destinations:** GET /api/locations/destinations — Returns distinct destinations currently present in the database.
- **Bookings:** GET /api/bookings and POST /api/bookings — Manages customer reservations.

Security & Performance Highlights in Backend:

- **SQL Injection Defense:** Parameterized Queries: Used db.query('SELECT * FROM hotels WHERE id = \$1', [id]) to prevent SQL Injection attacks.
- **Cross-Origin Security:** CORS Configuration: Allows authorized origin communication between localhost:5173 (Vite) and localhost:5000 (Express).
- **Body Parser Limit:** Payload Limit: Express JSON body parser set to 50MB to gracefully accept base64 image uploads.

6. Top 25 Technical Interview Questions & Answers

Q1: Can you give a brief walk-through of your project?

Answer: Stayora is a full-stack hotel booking platform built using React, Node.js, Express, and PostgreSQL. It has two main user personas: guests and administrators. Guests can search and filter hotels across Indian destinations, view detailed amenities and room options, explore hotel locations on an embedded OpenStreetMap, and book stays. Administrators have a dedicated CRUD dashboard to create, view, update, and delete hotels with real-time database synchronization.

Ø=Ü; Tip to say: "I focused on building a clean 3-tier architecture with full CRUD capability and interactive GIS map integration."

Q2: Why did you use PostgreSQL instead of MongoDB?

Answer: Hotels, rooms, and bookings have clearly defined, structured relationships. For example, rooms and bookings reference hotel records with relational foreign keys. PostgreSQL provides strong ACID compliance, ensuring that room counts and reservations are consistent without race conditions. It also natively supports array types for amenities and high-precision numeric types for latitude and longitude.

Ø=Ü; Tip to say: "Relational data with structured schemas and transaction consistency was a natural fit for PostgreSQL."

Q3: How did you implement the CRUD operations for hotels?

Answer: On the backend, I built RESTful endpoints in Express routed to hotelController and HotelModel. POST /api/hotels inserts a new row, GET /api/hotels fetches all rows with optional query filters, PUT /api/hotels/:id updates fields, and DELETE /api/hotels/:id deletes the record using parameterized SQL. On the frontend, ManageHotels.jsx consumes these APIs using fetch(), with immediate UI state updates and feedback alerts.

Ø=Ü; Tip to say: "I created both backend REST routes and frontend forms/tables with full input validation."

Q4: How did you integrate maps into the hotel cards and details page?

Answer: Instead of using expensive proprietary APIs, I used OpenStreetMap. Every hotel record stores latitude and longitude in PostgreSQL. On the frontend, I dynamically construct an OpenStreetMap iframe embed URL with a bounding box (bbox) and marker centered on the coordinates: 'https://www.openstreetmap.org/export/embed.html?bbox={lng-delta},{lat-delta},{lng+delta},{lat+delta}&marker={lat},{lng}'.

Ø=Ü; Tip to say: "It is lightweight, privacy-friendly, works out of the box, and doesn't require API keys."

Q5: What is the role of React Router in your project?

Answer: React Router (v6) enables Single Page Application (SPA) navigation without page reloads. I used BrowserRouter, Routes, and Route in App.jsx to configure routes like '/', '/hotels', '/hotels/:id', '/manage-hotels', and '/register'. I used useNavigate() for programmatic redirection (such as passing hotel details to the registration form) and useParams() to extract the hotel ID from the URL in HotelDetails.jsx.

Ø=Ü; Tip to say: "It enables fast, client-side routing while keeping URL states synchronized."

Q6: What is the difference between state and props in React?

Answer: State is internal data managed within a component that can change over time (e.g., the hotels array or search query in hotel.jsx). When state updates via useState, the component re-renders. Props (properties) are read-only data passed from a parent component to a child component (e.g., passing hotel and text props from hotel.jsx to HotelCard.jsx).

Ø=Ü; Tip to say: "State is internal and mutable; props are external and immutable."

Q7: How do you handle asynchronous operations in React?

Answer: I use JavaScript async/await syntax inside useEffect hooks. Because useEffect callback functions cannot directly be async, I declare an internal async function (e.g., fetchHotels) and invoke it. I also use Promise.all() or Promise.allSettled() when fetching multiple resources simultaneously (like hotels, locations, and rooms) to improve initial page load performance.

Ø=Ü; Tip to say: "Always handle loading states and catch errors gracefully inside async functions."

Q8: How do you protect your backend against SQL Injection?

Answer: I strictly avoid string concatenation like `SELECT \* FROM hotels WHERE id = ' + id`. Instead, I use parameterized queries provided by the 'pg' library: `db.query('SELECT \* FROM hotels WHERE id = \$1', [id])`. The PostgreSQL engine compiles the query template first, treating parameters strictly as literal values rather than

executable SQL code.

Ø=Ü; Tip to say: "Parameterized queries guarantee that user input can never alter query logic."

7. Additional Interview Questions (Advanced & Practical)

Q9: What is CORS and how did you configure it?

Answer: CORS stands for Cross-Origin Resource Sharing. By default, browsers block frontend applications (running on `http://localhost:5173`) from requesting APIs on a different origin or port (`http://localhost:5000`). In `server.js`, I imported the `'cors'` middleware in Express and configured allowed origins to safely permit frontend requests while preventing unauthorized domain access.

Ø=Ü; Tip to say: *"CORS is a browser security mechanism; Express middleware permits legitimate cross-port communication."*

Q10: How did you handle image uploads for hotels?

Answer: In the frontend hotel management form, I implemented a file input listener using the `FileReader` API (`readAsDataURL``). This converts the user's selected image file into a base64-encoded string previewable immediately in the UI. When submitted, the string is sent in the JSON payload. To prevent 'Payload Too Large' (HTTP 413) errors, I configured Express body-parser with a 50MB limit: `express.json({ limit: '50mb' })``.

Ø=Ü; Tip to say: *"FileReader provides instant client preview and Express limits accommodate image payloads."*

Q11: What HTTP status codes did you use in your API?

Answer: I used standard REST HTTP status codes: 200 OK for successful GET and PUT requests; 201 Created when a hotel or booking is successfully inserted; 400 Bad Request if mandatory fields are missing; 404 Not Found if a requested hotel ID does not exist; and 500 Internal Server Error for unhandled database exceptions.

Ø=Ü; Tip to say: *"Using correct status codes helps the frontend respond intelligently to API outcomes."*

Q12: How does the hotel search and filter functionality work?

Answer: It is implemented both client-side and server-side. In the frontend, `hotel.jsx` has a filter function checking if the search keyword is included in the hotel's name, city, state, or destination. On the backend, `HotelModel.findAll()` supports query parameters like `?destination=Goa` or `?search=Kochi`, appending SQL conditions with ``ILIKE`` or ``LOWER()`` for case-insensitive matching.

Ø=Ü; Tip to say: *"Dual-layer filtering ensures responsive typing feedback and efficient database querying."*

Q13: What happens step-by-step when a user books a room?

Answer: 1. Guest clicks 'Book Now' on a hotel or room card. 2. The hotel name and room type are saved in `state/localStorage`, and user is navigated to `/register`. 3. Guest fills out traveler information and payment method. 4. Clicking 'Confirm' sends `POST /api/bookings` to the backend. 5. Backend generates a unique booking ID (e.g. `STAY-1710000001`) and saves it to PostgreSQL. 6. Guest is redirected to `/confirmation` displaying the full receipt.

Ø=Ü; Tip to say: *"It is an end-to-end multi-step flow with input validation and database persistence."*

Q14: What is connection pooling and why is it used in PostgreSQL?

Answer: Instead of opening and closing an expensive TCP database connection for every single HTTP request, a connection pool (via `pg.Pool``) creates a reusable pool of active connections. When an incoming request arrives, a client is leased from the pool, executes the query, and is released back to the pool immediately. This drastically reduces connection latency and CPU overhead.

Ø=Ü; Tip to say: *"Pools maximize throughput and prevent database overload under concurrent traffic."*

Q15: What challenges did you face during development and how did you resolve them?

Answer: One challenge was adding GPS coordinates to existing database records without breaking existing queries. I resolved this by adding idempotent migrations (``ALTER TABLE hotels ADD COLUMN IF NOT EXISTS latitude NUMERIC(10, 6)``) and updating both schema and seed data. Another challenge was payload size errors when users uploaded photos, which I resolved by increasing Express body limits to 50MB and adding a 5MB client validation check.

Ø=Ü; Tip to say: *"Always mention the problem, your debugging steps, and the concrete engineering fix."*

Q16: If you had more time, what features would you add next?

Answer: I would implement user authentication using JWT (JSON Web Tokens) or OAuth, integrate a live payment gateway like Razorpay or Stripe, add automated email notifications on booking confirmation using Nodemailer, and deploy the application on AWS or Render with PostgreSQL hosted on Supabase or Neon.

Ø=Ü; Tip to say: *"Showing future vision demonstrates an engineering mindset and awareness of production systems."*

8. HR & Behavioral Interview Tips

1. 'Tell me about yourself'

"I am a passionate Full-Stack Developer with strong foundations in React, Node.js, Express, and PostgreSQL. I enjoy building end-to-end web applications that solve practical problems. Recently, I developed Stayora, a hotel booking and CRUD management application with interactive map integration and dynamic database querying. I am eager to contribute my problem-solving skills to your development team."

2. Key Rules for Acing the Technical Interview:

- **Rule 1:** Do not memorize blindly: Understand WHY you chose a tool (e.g., PostgreSQL for relational integrity, OpenStreetMap for zero-cost coordinates).
- **Rule 2:** Speak with clarity: Break down your answers into 1) What it is, 2) How you built it, and 3) The result.
- **Rule 3:** Be honest about bugs: When asked about challenges, talk about errors you encountered (like body-parser size limits or SQL column migrations) and how you solved them.
- **Rule 4:** Demonstrate code awareness: Mention file names like ManageHotels.jsx, HotelDetails.jsx, HotelModel.js, and schema.sql to show you actually wrote the code.

